THEORY ASSIGNMENT - 3

Subject: Frontend Development

Name: Prateek Jadaun Roll No.: 32 Section: NB

Submission Date: 22/10/2026

Q1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript runs in a web browser and makes a webpage interactive. Backend JavaScript runs on a server, usually with Node.js, and handles data, APIs and other server operations.

Frontend JavaScript	Backend JavaScript
Runs in the browser.	Runs on the server.
Works with the HTML page and DOM.	Works with server logic, APIs and databases.
Example: checking a signup form.	Example: saving signup details to a database.

Q2. What is DOM?

DOM stands for Document Object Model. It represents an HTML page as a tree of objects. JavaScript uses the DOM to read or change webpage elements, text and styles.

```
<p id="demo">Hello</p>
<script>
  document.getElementById("demo").textContent = "Welcome";
</script>
```

Here the paragraph text changes from Hello to Welcome.

Q3. What is an event in JavaScript?

An event is an action that takes place on a webpage, such as a button click, key press, input change or form submission. JavaScript can run code in response to these actions.

```
<button onclick="alert('Button clicked!')">Click Me</button>
```

When the button is clicked, an alert message appears.

Q4. What is the event loop in JavaScript?

The event loop helps JavaScript manage asynchronous work. Normal code runs on the call stack. When the stack is empty, the event loop allows ready tasks from the queues to execute. This helps the webpage stay responsive.

```
console.log("Start");
setTimeout(() => console.log("Hello"), 1000);
console.log("End");
// Output: Start, End, Hello
```

The timer callback runs later, so End is printed before Hello.

Q5. What is hoisting in JavaScript?

Hoisting means JavaScript processes declarations before executing the code in their scope. A function declaration can be called before its definition. A variable declared with var exists before its assignment but has the value undefined. Variables declared using let and const cannot be accessed before their declaration.

```
console.log(a);  // undefined
var a = 10;
```

Q6. What is event bubbling?

Event bubbling happens when an event on a child element also travels upward through its parent elements. For example, clicking a button inside a div can trigger both click handlers.

```
<div onclick="alert('Div clicked')">
  <button onclick="alert('Button clicked')">Click</button>
</div>
```

The button alert appears first, followed by the div alert.

Q7. What is the difference between named function, anonymous function and arrow function?

A named function has its own name. An anonymous function is written without a name and is often assigned to a variable or passed as an argument. An arrow function uses shorter syntax with => and handles this differently from a regular function.

```
function greet1() { console.log("Hello"); }       // Named
const greet2 = function() { console.log("Hello"); }; // Anonymous
const greet3 = () => console.log("Hello");         // Arrow
```

All three examples can print Hello when their function is called.

Q8. Explain for-of and for-in loops.

A for-of loop reads values from an iterable such as an array or a string. A for-in loop reads enumerable property keys, so it is mainly useful for objects.

```
let numbers = [10, 20, 30];
for (let value of numbers) console.log(value);
// 10 20 30

let student = {name: "Rahul", age: 20};
for (let key in student) console.log(key, student[key]);
// name Rahul
// age 20
```

Q9. Explain any five array methods.

Arrays have built-in methods for adding, removing and processing elements. Five useful methods are:

• push(): Adds an element at the end of an array.

• pop(): Removes the last element.

• shift(): Removes the first element.

• unshift(): Adds an element at the beginning.

• map(): Makes a new array by applying a function to each element.

```
let a = [10, 20];
```

```
a.push(30);          // [10, 20, 30]
a.pop();             // [10, 20]
a.shift();           // [20]
a.unshift(5);        // [5, 20]
let b = a.map(x => x * 2); // [10, 40]
```

Q10. What is a callback method?

A callback is a function passed to another function as an argument. The receiving function calls it when needed. Callbacks are commonly used with events, timers and array methods.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
function message() {
  console.log("Welcome to JavaScript");
}
greet("Rahul", message);
// Hello Rahul
// Welcome to JavaScript
```

In this example, message is the callback function passed to greet.